

2026 €€€ 超专业级别硬件能力认证第一轮
(C5P-S1) 提高级 C++语言试题

认证时间：2026 年 9 月 18 日 14:30~16:30

考生注意事项：

- 试题纸共有 14 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 下列 Linux 命令中，用于连接并显示文件内容的是（ ）
A. cp
B. cat
C. ping
D. touch
2. 已知某用二叉链表存储的哈夫曼树共有 m 个叶子结点，则该哈夫曼树中总共有多少个空指针域（ ）
A. m
B. $m+1$
C. $2m$
D. $2m+1$
3. 对于一棵包含 n 个结点的完全二叉树，对每个叶子结点向上访问直到根结点，统计路径上经过的结点数之和。总时间复杂度为（ ）
A. $O(n)$
B. $O(n \log n)$
C. $O(n^2)$
D. $O(n^2 \log n)$
4. 小 h 从原点 $(0,0)$ 出发，每一步向右或向上走，到达点 $(5,5)$ ，且始终满足 $x \geq y$ 。满足条件的路径有多少条（ ）
A. 252
B. 90
C. 132
D. 42
5. 用两个栈模拟一个队列，入队时元素统一压入栈 A。正确的出队操作是（ ）
A. B 为空时将 A 全部转移到 B，再从 B 弹出；B 非空时直接从 B 弹出
B. 无论 B 是否为空，都先将 A 全部转移到 B，再从 B 弹出

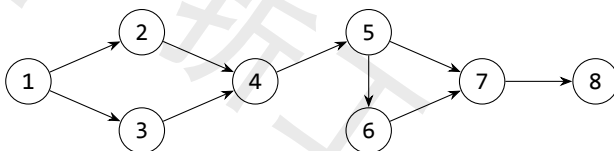
- C. 直接删除 A 的栈底元素
D. 直接从 A 弹出栈顶元素
6. 从集合 $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ 中选出 4 个不同的数, 要求任意两个被选数都不相邻。不同的选法共有 ()
A. 15
B. 20
C. 35
D. 70
7. 在 C++ 中, 关于 `std::set<int>` 与 `std::map<int, int>` 的说法, 正确的是 ()
A. `set` 中的元素可以重复
B. `map` 按插入顺序存储键值对
C. `set` 自动有序, 插入、删除、查找复杂度均为 $O(\log n)$
D. `map` 的键可以重复
8. 关于有向无环图的拓扑排序, 下列说法正确的是 ()
A. 任何有向无环图的拓扑排序都唯一
B. 含有环的有向图也可以进行拓扑排序
C. 若有边 $u \rightarrow v$, 则任一拓扑序中 u 在 v 之前
D. 拓扑排序就是深度优先遍历序列
9. KMP 算法中, `next` 数组的主要作用是 ()
A. 记录模式串失配后应回溯到的位置
B. 记录主串每个字符出现的次数
C. 记录主串的后缀数组
D. 与模式串无关地跳过主串位置
10. 对一个 n 个顶点、 m 条边的带权有向图使用不带堆优化的 Dijkstra 算法, 时间复杂度为 ()
A. $O(n^2)$
B. $O(m \log n)$
C. $O((m+n) \log n)$
D. $O(mn)$
11. 已知 20260913 是质数, 表达式 $(1/2 + 1/3 + \dots + 1/20260912)^{20260913} \bmod 20260913$ 的值为 ()
A. 0
B. 1
C. $2^{20260913} \bmod 20260913$
D. 20260912
12. 设 $p = 20260913$ 为质数。在模 p 意义下,

$$\sum_{i=1}^{p-2} \frac{1}{i(i+1)}$$

等于 ()

- A. 0
- B. 1
- C. 2
- D. $p - 2$

13. 对下图恰好删去两条不同的边, 使得仍存在从 1 到 8 的有向路径。删边方案共有 () 种



- A. 12
- B. 15
- C. 18
- D. 21

14. 1 到 200 之间, 不能被 2、3、7 中任何一个整除的整数共有 () 个

- A. 57
- B. 58
- C. 59
- D. 60

15. 已知如下 C++14 代码定义 Ackermann 函数:

```
01 long long A(int m, long long n) {  
02     if (m == 0) return n + 1;  
03     if (n == 0) return A(m - 1, 1);  
04     return A(m - 1, A(m, n - 1));  
05 }
```

则二元组 $(A(3, 3), A(3, 10^{18}) \bmod 101)$ 的值为 ()

- A. (61, 5)
- B. (61, 8)
- C. (31, 5)
- D. (31, 8)

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 √，错误填 ×；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

(1) 给定一个非空字符串，程序先求出每个前缀的最长真前缀与后缀公共长度，再把整个字符串以及它的各级相同前后缀的长度相加并输出。输入字符串只含小写英文字母。

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 2e5 + 5;
04int n, pi[N];
05string s;
06
07int main() {
08    cin >> s;
09    n = s.size();
10    for (int i = 1; i < n; ++i) {
11        int j = pi[i - 1];
12        while (j && s[i] != s[j]) j = pi[j - 1];
13        if (s[i] == s[j]) ++j;
14        pi[i] = j;
15    }
16    int ans = 0;
17    for (int len = n; len > 0; len = pi[len - 1]) ans += len;
18    cout << ans << '\n';
19}
```

• 判断题

16. 当输入字符串为 ababa 时，程序计算得到 $pi[4]=3$ 。()

- A. 正确
- B. 错误

17. 若把累加长度循环的初始化语句 $len = n$ 改为 $len = pi[n-1]$ ，对任意输入，程序输出都恰好减少 n 。()

- A. 正确
- B. 错误

18. 若把求解 pi 的循环中的 `while` 改成 `if`，对所有输入，程序输出仍然不变。()

- A. 正确
- B. 错误

• 单选题

19. 输入 abacababacabab 时, 程序输出为 ()

- A. 16
- B. 20
- C. 24
- D. 28

20. (4 分) 若输入字符串长度为 n , 且字符比较、数组访问均视为 $O(1)$, 则程序的最坏时间复杂度和空间复杂度分别为 ()

- A. 时间 $O(n^2)$, 空间 $O(n)$
- B. 时间 $O(n)$, 空间 $O(n)$
- C. 时间 $O(n \log n)$, 空间 $O(n)$
- D. 时间 $O(n)$, 空间 $O(1)$

(2) 给定一个有 n 行、 m 列的棋盘, 字符 # 表示不能放置, 其余位置可以放置。要求同一行相邻位置、相邻两行同一列的位置不能同时放置, 程序求合法方案数。

```
01#include <bits/stdc++.h>
02using namespace std;
03const int P = 998244353, M = 10;
04int n, m, ban[105], f[2][1 << M];
05vector<int> st;
06
07int main() {
08    cin >> n >> m;
09    for (int i = 1; i <= n; ++i) {
10        string s; cin >> s;
11        for (int j = 0; j < m; ++j)
12            if (s[j] == '#') ban[i] |= 1 << j;
13    }
14    for (int s = 0; s < (1 << m); ++s)
15        if ((s & (s << 1)) == 0) st.emplace_back(s);
16    f[0][0] = 1;
17    for (int i = 1; i <= n; ++i) {
18        int pre = i & 1, cur = pre ^ 1;
19        int mask = pre ^ cur;
20        pre ^= mask;
21        cur ^= mask;
22        memset(f[cur], 0, sizeof(f[cur]));
23        for (auto&& s : st) {
```

```

24         if (s & ban[i]) continue;
25         for (auto&& t : st) {
26             if (s & t) continue;
27             f[cur][s] += f[pre][t];
28             if (f[cur][s] >= P) f[cur][s] -= P;
29         }
30     }
31 }
32 int ans = 0;
33 for (auto&& s : st) {
34     ans += f[n & 1][s];
35     if (ans >= P) ans -= P;
36 }
37 cout << ans << '\n';
38 }

```

• 判断题

21. 当 $m=3$ 时, 经过第一个双重循环后, `st.size()` 的值为 5。()
- A. 正确
- B. 错误
22. 在某次转移中, 若当前状态 $s=101$ 、上一行状态 $t=001$, 则程序会执行 `continue`, 不会把 `f[pre][t]` 加入当前状态。()
- A. 正确
- B. 错误
23. (2 分) 若删去每轮开始时的 `memset(f[cur], 0, sizeof(f[cur]))`, 程序对所有输入的输出仍不变。()
- A. 正确
- B. 错误

• 单选题

24. 记 $V = st.size()$, 则程序的时间复杂度为 ()
- A. $O(nV)$
- B. $O(nV^2 + 2^m)$
- C. $O(n2^{2m})$
- D. $O(n^2V)$
25. 输入为 $n=2, m=3$, 两行均为 ... 时, 程序输出为 ()
- A. 15
- B. 16
- C. 17

D. 18

26. 若删去主循环中的两条语句 `pre = mask;` 和 `cur = mask;`, 程序输出将 ()

A. 只有输入的第一行会受到影响, 后续结果不变

B. 输出为 0

C. 仅当棋盘中有不能放置的位置时才会改变

D. 与原程序完全相同

(3) 给定一个长度为 n 的数组, 操作 1 $l\ r\ k\ b$ 把区间内每个数变为 $kx + b$, 操作 2 $l\ r$ 输出区间和。所有区间下标均合法。

```
01#include <bits/stdc++.h>
02using namespace std;
03using ll = long long;
04const ll P = 1000000007;
05const int N = 2e5 + 5;
06int n, q;
07ll a[N];
08struct Node { ll sum, mul, add; } tr[N << 2];
09
10void build(int p, int l, int r) {
11    tr[p].mul = 1;
12    if (l == r) { tr[p].sum = a[l] % P; return; }
13    int m = (l + r) >> 1;
14    build(p << 1, l, m); build(p << 1 | 1, m + 1, r);
15    tr[p].sum = (tr[p << 1].sum + tr[p << 1 | 1].sum) % P;
16}
17void apply(int p, int l, int r, ll k, ll b) {
18    tr[p].sum = (tr[p].sum * k + (r - l + 1) * b) % P;
19    tr[p].mul = tr[p].mul * k % P;
20    tr[p].add = (tr[p].add * k + b) % P;
21}
22void push(int p, int l, int r) {
23    if (l == r) return;
24    int m = (l + r) >> 1;
25    apply(p << 1, l, m, tr[p].mul, tr[p].add);
26    apply(p << 1 | 1, m + 1, r, tr[p].mul, tr[p].add);
27    std::exchange(tr[p].mul, 1);
28    std::exchange(tr[p].add, 0);
```

```

29}
30void upd(int p, int l, int r, int ql, int qr, ll k, ll b)
    {
31    if (ql <= l && r <= qr) { apply(p, l, r, k, b); return
        ; }
32    push(p, l, r);
33    int m = (l + r) >> 1;
34    if (ql <= m) upd(p << 1, l, m, ql, qr, k, b);
35    if (qr > m) upd(p << 1 | 1, m + 1, r, ql, qr, k, b);
36    tr[p].sum = (tr[p << 1].sum + tr[p << 1 | 1].sum) % P;
37}
38ll qry(int p, int l, int r, int ql, int qr) {
39    if (ql <= l && r <= qr) return tr[p].sum;
40    push(p, l, r);
41    int m = (l + r) >> 1;
42    ll s = 0;
43    if (ql <= m) s = (s + qry(p << 1, l, m, ql, qr)) % P;
44    if (qr > m) s = (s + qry(p << 1 | 1, m + 1, r, ql, qr)
        ) % P;
45    return s;
46}
47int main() {
48    cin >> n >> q;
49    for (int i = 1; i <= n; ++i) cin >> a[i];
50    build(1, 1, n);
51    while (q--) {
52        int op, l, r; cin >> op >> l >> r;
53        if (!(op ^ 1)) {
54            ll k, b; cin >> k >> b;
55            upd(1, 1, n, l, r, k, b);
56        } else cout << qry(1, 1, n, l, r) << '\n';
57    }
58}

```

• 判断题

27. 在执行一次 upd 时, 如果当前结点区间完全包含在更新区间内, 程序会调用一次 apply 后直接返回, 不再访问该结点的子结点。()

- A. 正确
B. 错误
28. 若把 `push` 末尾的 `tr[p].add` 重置语句删去, 程序对所有输入的输出仍然不变。()
A. 正确
B. 错误
29. 程序的时间复杂度为 $O((n+q)\log n)$ 。()
A. 正确
B. 错误
- 单选题
30. 设当前结点 p 表示区间 $[l, r]$, 执行 `apply(p, l, r, k, b)` 后, `tr[p].sum` 的新值为 ()
A. $(tr[p].sum * k + b) \% P$
B. $(tr[p].sum * k + (r - l + 1) * b) \% P$
C. $((tr[p].sum + b) * k) \% P$
D. $((r - l + 1) * (tr[p].sum * k + b)) \% P$
31. 关于函数 `push`, 下列修改中不会改变程序输出的是 ()
A. 交换第一个、第二个 `apply` 调用 (左、右子结点) 的先后顺序
B. 在调用两个子结点的 `apply` 前, 先把 `tr[p].add` 置为 0
C. 把两个 `apply` 调用中的参数 `b` 都改为 0
D. 删除右子结点的 `apply` 调用, 但保留左子结点的调用
32. (4 分) 初始数组为 2 1 3 5 4 6, 依次执行 1 2 6 2 1、1 1 4 3 2、1 3 5 1 4、2 2 6, 最后一次操作输出为 ()
A. 99
B. 103
C. 105
D. 107

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (三色序列) 给定一个长度为 n 、只含字符 R,G,Y 的字符串。每次可以交换两个相邻字符。求使相邻字符均不同所需的最少交换次数；若无法做到，输出 -1。 $1 \leq n \leq 400$ 。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 405, INF = 1e9;
04 int n, cnt[3], pos[3][N], pre[3][N];
05 int f[2][N][N][4];
06 string s;
07 int main() {
08     cin >> n >> s;
09     for (int i = 1; i <= n; ++i) {
10         int c;
11         if (s[i-1] == 'R') c = 0;
12         else if (s[i-1] == 'G') c = 1;
13         else c = 2;
14         for (int t = 0; t < 3; ++t)
15             pre[t][i] = ①;
16         pos[c][++cnt[c]] = i;
17         ++pre[c][i];
18     }
19     memset(f, 0x3f, sizeof(f));
20     f[0][0][0][3] = ②;
21     for (int len = 0; len < n; ++len) {
22         int cur = len & 1, nxt = cur ^ 1;
23         memset(f[nxt], 0x3f, sizeof(f[nxt]));
24         for (int r = 0; r <= cnt[0]; ++r)
25             for (int g = 0; g <= cnt[1]; ++g) {
26                 int y = ③;
27                 if (y < 0 || y > cnt[2]) continue;
28                 int used[3] = {r, g, y};
29                 for (int last = 0; last < 4; ++last) {
30                     int val = f[cur][r][g][last];
31                     if (val >= INF) continue;
32                     for (int c = 0; c < 3; ++c) {
```

```

33         if ( ④ ) continue;
34         int x = pos[c][used[c]+1];
35         int w = x-len-1;
36         for (int t = 0; t < 3; ++t)
37             w += max(0,used[t]-pre[t][x]);
38         int nr = r, ng = g;
39         if (c == 0) ++nr;
40         if (c == 1) ++ng;
41         int nv = ⑤;
42         f[nxt][nr][ng][c] =
43             min(f[nxt][nr][ng][c], nv);
44     }
45 }
46 }
47 }
48 int ans = INF, z = n & 1;
49 for (int c = 0; c < 3; ++c) {
50     int v = f[z][cnt[0]][cnt[1]][c];
51     ans = min(ans, v);
52 }
53 cout << (ans >= INF ? -1 : ans) << '\n';
54 }

```

33. ①处应填 ()
 A. pre[t][i-1] B. pre[t][i] + 1
 C. pre[t][i-1] + (t == c) D. pre[c][i-1]
34. ②处应填 ()
 A. INF B. 0
 C. 1 D. len
35. ③处应填 ()
 A. len-r-g B. r+g-len
 C. n-r-g D. cnt[2]-r-g
36. ④处应填 ()
 A. c == last || used[c] == cnt[c] B. c != last && used[c] < cnt[c]
 C. c == last && used[c] == cnt[c] D. c != last || used[c] < cnt[c]
37. ⑤处应填 ()
 A. val B. w

C. $val + w$ D. $val + len - w$

(2) (盒子) 有 n 个体积互不相同的盒子和 k 个物品。每个盒子中恰放一个物品或一个体积更小的盒子；把体积为 x 的盒子放入体积为 y 的盒子需要 $y - x$ 单位缓冲材料。所有盒子均须使用。先求所需缓冲材料的最小总量；随后有 q 次删除，每次永久删除一个指定盒子并再次求答案。保证始终至少剩下 k 个盒子。 $1 \leq n \leq 2 \times 10^5$ 。

```
01#include <bits/stdc++.h>
02using namespace std;
03using ll = long long;
04const int N = 2e5 + 5;
05int n, k, q, need;
06ll c[N], ans;
07set<ll> box;
08multiset<ll> lo, hi;
09
10void fix() {
11    while ((int)lo.size() > need) {
12        auto it = ①;
13        ans -= *it; hi.insert(*it); lo.erase(it);
14    }
15    while ((int)lo.size() < need) {
16        auto it = hi.begin();
17        ans += *it; lo.insert(*it); hi.erase(it);
18    }
19    while (!lo.empty() && !hi.empty()) {
20        auto x = prev(lo.end()), y = hi.begin();
21        if (②) break;
22        ll vx = *x, vy = *y;
23        ans += ③;
24        lo.erase(x); hi.erase(y);
25        lo.insert(vy); hi.insert(vx);
26    }
27}
28void add_gap(ll x) { lo.insert(x); ans += x; }
29void del_gap(ll x) {
30    auto it = lo.find(x);
31    if (it != lo.end()) { ans -= x; lo.erase(it); }
```

```

32     else hi.erase(hi.find(x));
33 }
34 int main() {
35     cin >> n >> k;
36     for (int i = 1; i <= n; ++i) {
37         cin >> c[i];
38         box.insert(c[i]);
39     }
40     if (box.size() > 1) {
41         auto x = box.begin(), y = next(x);
42         for (; y != box.end(); ++x, ++y)
43             add_gap(*y - *x);
44     }
45     need = ④;
46     fix(); cout << ans << '\n';
47     cin >> q;
48     while (q--) {
49         int d; cin >> d;
50         ll v = c[d], l = 0, r = 0;
51         auto it = box.find(v);
52         auto rt = next(it);
53         bool hl = it != box.begin();
54         bool hr = rt != box.end();
55         if(hl){ l=*prev(it); del_gap(v-l); }
56         if(hr){ r=*rt; del_gap(r-v); }
57         box.erase(it);
58         --need;
59         if (hl && hr) add_gap(⑤);
60         fix(); cout << ans << '\n';
61     }
62 }

```

38. ①处应填 ()

- A. lo.begin() B. prev(lo.end())
 C. hi.begin() D. prev(hi.end())

39. ②处应填 ()

- A. *x < *y B. *x <= *y

C. $*x > *y$ D. $*x \geq *y$

40. ③处应填 ()

A. $vx - vy$ B. $vy - vx$

C. $vx + vy$ D. $-vx$

41. ④处应填 ()

A. $n - k$ B. $k - 1$

C. $n - 1$ D. k

42. ⑤处应填 ()

A. $v - 1$ B. $r - v$

C. $r - 1$ D. $r + 1 - v$